

- Embedded system

- ① Processor ✓
- ② Memory
- ③ I/O Peripheral.

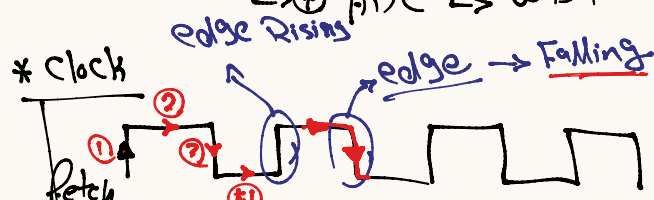
MP	MC
AVR	Atmega32 Atmega128 Atmega16
ARM	STM32F103 STM32F411RE

* clock

MC → Digital circuit → sequentially
 → need the periodic wave to move from state to another
 → Periodic wave called clock

→ type Digital circuit

- ① CPU → ③ UART
- ② timer → ④ SPI
- ⑤ I2C → ⑥ PWM
- ⑦ ADC → ⑧ WDT



→ for each transition → sync with edge
 [Falling: clk move from High to Low]
 [Rising: clk move from Low to High]

Based on design for Peripheral or CPU

* clock → select to speed operation for digital circuit or CPU & Peripheral

→ For ex: freq = 8 MHz →
 Periodic = $\frac{1}{\text{freq}} = \frac{1}{8\text{M}} = \frac{1}{8\text{M}}$

→ clock → in one sec

→ 8000000 / sec

- ① clk → 1 Instruction
- 8M clk → 8 Million IS / sec
- 16M clk / s → 16

→ clock source ✓

→ clock frequency ✓

→ clock distribution

→ CPU frequency

→ Peripheral clock

* clock source

→ it's component to generate wave for MC.

→ clock source ≠ clock frequency
 who generate clock | what is speed for clock?

→ Source:

→ ① Internal RC oscillator

→ inside MC → oscillator Based on RC circuit
 → Resistor & Cap



→ about this type Advantage

- * not need external component
- * low cost

→ RC = 8 MHz

dis Advantage
 * Low Accuracy
 * Frequency temp/voltage

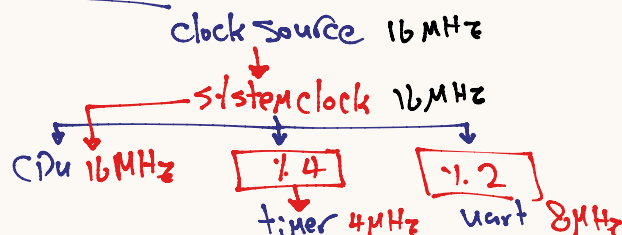
→ ② External crystal

→ use crystal out side MC
 → 8, 16, 20, 25 MHz
 → Accuracy & stability

→ ③ external clock

→ ④ low frequency crystal

clock tree (clock distribution)



* why clock is important

* ① Performance: clock ↑ ⇒ Instruction in sec
 8 MHz ⇒ 8000000 IPS
 16 MHz ⇒ 16M IPS

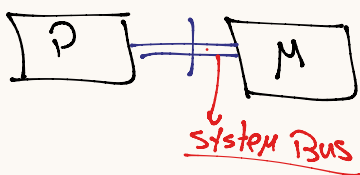
* ② Power → clock ↑ → Power consumption ↑

- System Bus in Embedded

- * it's Communication way between Processor & memories & I/O Peripherals
- * wire (SoC) | track (SoC) | tunnel (SoC)
- write Board PCB MC

* System Bus Contain:

- data Bus: transive for data
- Address Bus: transive Address
- Control Bus: state operation
- Read/write



* System Bus size

→ system bus have No of Bits

system bus → 32 bit →

Address 32 wire data 32 wire Control 32 wire

→ size of system bus effect on

- int Datatype
- Pointer size

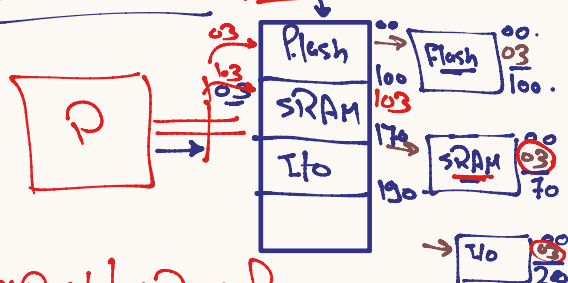
char * ptr = &y

Fixed size
Base on system bus



* System Bus Architecture

1) von. numman Memory Map (Main Memory)

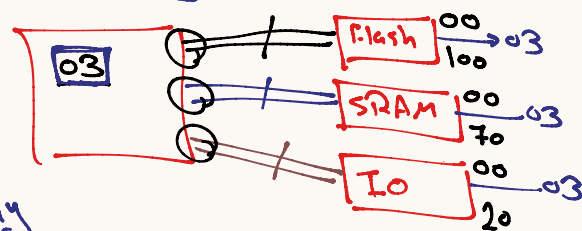


- * 1) system bus for all memory
- * 2) Memory Map
- * 3) H.w simple | Low cost
- * 4) low speed Access timing Accessing

- system Bus Architecture

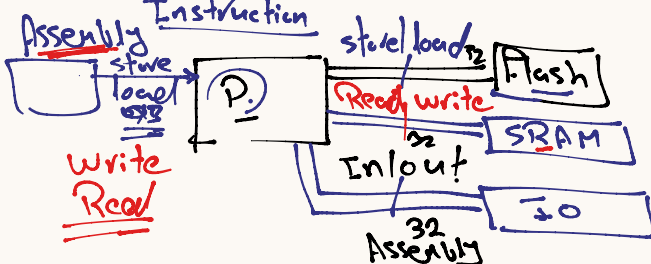
2) Harvard → (Port Map)

→ for each memory have specific system bus



* Assembly

→ for each system bus have specific



* Port Map → any system bus have Assembly instruction

* write Assembly → complex sw

* Complex H.w

* Note:

if write c_code → Assembly

int * ptr = specific Address;

default

→ Compiler → Assembly

if you write any

code to Access Address

in RAM Address

→ SRAM

write c Code

→ Flash / I/O

write Assembly

if you need write Assembly in c_code

→ asm -- volatile -- ("add %0, %1 mit:");

* Memory Map Address

Port Map Address

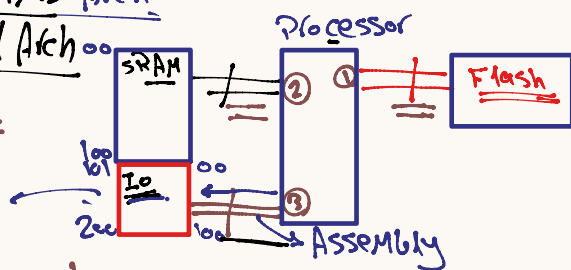
- C_code
- von numman
- 1 system bus.
- Map for Address

- Assembly
- Harvard
- Multi system bus
- Map / Port
- in/out
- write
- Read
- store/load

- AVR starter bus Arch

it's Harvard Arch

but not brid



* if you write C-Code → ②

* if you need go to Flash → Assembly

* if you need Access Address in I/O

C-Code
You will use Memory Map

Assembly
You will use
PORT Map

* the I/O Register have 2 Address

→ Memory → C

→ PORT → Assembly

- Atmega32

→ Atmel → microshin.

* How Access Address in Memory

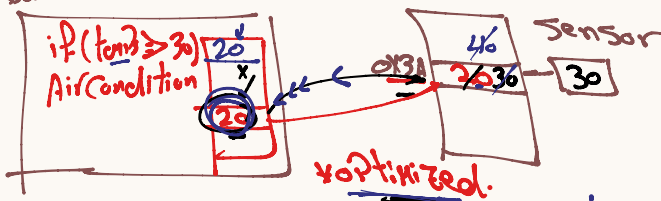
→ Address = 0x3A → u8

→ C-Code → to Access it
uint8_t *Ptr = (uint8_t *)0x3A;

Address → Casting

Point Div = No₁/No₂; → Casting
= (Point)/No₁/No₂;

→ 0x3A → store sensor Value



uint8_t *Ptr = (uint8_t *)0x3A;

*Ptr → value

volatile uint8_t *Ptr = (volatile uint8_t *)0x3A

*Ptr → value

* 0x5A → 30

* ((volatile uint8_t *)0x5A) = 30; ←

#define temp * ((volatile uint8_t *)0x5A)
volatile uint8_t * temp = (volatile uint8_t *)0x5A;

○ Prüfung

1